Lecture 2 - Case Study

Agile to the Rescue! (Right?)

Before jumping into planning the jukebox project, Joanna, Bruce, and Dan talked about how waterfall processes had caused problems for their teams in the past.

On their last project, Bruce and Dan worked with Tom, a customer account manager at the company, who spent a lot of time on the road helping customers at arcades, casinos, bars, and restaurants install and use their products. The three of them spent the first few weeks of the project building up a specification for the new slot machine. Tom was only in the office half the time, which gave Bruce and Dan time to start designing the software and planning the architecture. Once all three of them had agreed on the requirements, they called a big meeting with the CEO and senior managers of the company to review the requirements, make necessary changes, and get approval to start building.

At that point, Tom went back out on the road, leaving the work up to Bruce and Dan. They broke the project down into tasks, dividing them up among the team, and had everyone start building the software. When the team was almost done building the software, Tom gathered a group of business users, project managers, and executives into a big conference room to do a demo of the nearly complete Slot-o-matic Weekend Warrior software.

It didn't go nearly as smoothly as they'd expected.

At the demo, they had an awkward conversation where the CEO asked, "The software looks great, but wasn't it supposed have a video poker mode?" That was an unfortunate discovery. Apparently, the CEO was under the impression that they were working on software that could be deployed to either the slot machine hardware or the video poker hardware. There had been a lot of discussion of this between the senior managers, the board, and the owners of their two biggest customers. Too bad nobody had bothered to tell the team.

The worst part about it was that if Dan had known about this earlier in the project, it wouldn't have been difficult to change direction. But now, they had to rip out enormous amounts of code they'd already written, and replace it with code retrofitted from the video poker project. They spent weeks troubleshooting weird bugs that were caused by integration problems. Dan spent many late nights complaining to Bruce that this was 100% predictable, and that this almost always happens when code built for one job is hastily repurposed to do something else. Now he's going to be stuck with a tangled mess of spaghetti code. It's going to be difficult to maintain this codebase, and the team is frustrated because it clearly didn't have to be this way.

And this wasn't just a problem for Dan and Bruce. The project manager for that project was so unhappy that he left the company. He had trusted the team's estimates and status, which were completely destroyed by the video poker change. The team had no idea that they would have to deal with the unexpected hardware change, but that didn't make the project manager's life any easier. Even though the facts on the ground had changed, the deadline was set in stone. By the time the project ended, the plan was out of date and basically useless, but the project manager was being held accountable for it anyway. After being raked over the coals by the senior managers in the company, all he could say in his defense is that his team didn't give good numbers to start with. Pretty soon, he left the company for another job—and Joanna was hired shortly after that.

Tom may have been the most frustrated of the group, because he was the one who had to face the customers when they ran into problems. Their biggest customer for this product was the Little Rock, a casino in Las Vegas that wanted to customize all of their slot machines with themes to match their reproductions of cities in Arkansas. The customer had requested this new feature so they could change games between shifts without having to move kiosks around. Their engineers kept running into bugs left in the software by Bruce's team, which meant that Tom and Dan had to spend weeks on the phone with the engineers coming up with patches and workarounds. The Little Rock didn't cancel any contracts, but their next big order was with a competitor, and it wasn't a secret that the CEO and managers blamed the Slot-o-matic Weekend Warrior project for the loss of business.

In the end, everyone knew that the project went wrong. And each person had a pretty good idea of how it was somebody else's fault. But nobody seemed to have any idea how to fix these types of recurring problems. And the software that they'd delivered was still a mess.

Adding Agile Makes a Difference

Bruce, Dan, and Joanna took Tom out to lunch the next time he was in town. After spending time commiserating about their past project problems, Joanna suggested that it was time to go agile.

Like many teams beginning their move to agile, they started with a discussion about what the word "agile" really meant to each of them. To Bruce, it just referred to the world of agile development: the specific books, practices, training courses, blogs, and people who practice agile. To Joanna, agile specifically meant "the ability for a project to handle changes," and it was mainly a set of practices focused on that goal. Dan thought agile meant not writing any documentation, and just jumping straight into code. And Tom had no idea what they were talking about, but he was happy that they were talking about giving him lots of demos along the way, so he could avoid what happened last time.

Members of a team that has started to "go agile" typically start educating themselves on agile techniques, practices, and ideas, and this team was no exception. Dan joined the Agile Alliance and started connecting with other agile practitioners. Bruce and Joanna started reading blogs and books about agile development and project management. Both of them saw great ideas, and took it upon themselves to use what they learned to start anticipating and fixing their project's problems. Each of them discovered different things, and immediately started adding them into the mix.

Dan had already written automated unit tests for his previous projects, but a lot of the developers on the jukebox project had never written any. He started working with the developers to do unit testing and **test-driven development**. He created an **automated build script**, and set up a **build server** that would check out the code, build the software, and then run the tests once an hour. And it worked! They immediately saw an improvement in their code. Every day a developer found a bug that they never would have caught without the automated tests, and it was clear that they were avoiding weeks of debugging and tracking down nasty problems in the field. Not only were they creating fewer bugs, they also felt like the code they were building was easier to change.

Joanna attended **Scrum** training, and now the team has taken to calling her the Scrum Master (although she learned in her training that there's a big difference between a Scrum Master and a project manager, and she isn't 100% sure that the role that she's playing on the project really deserves to be called Scrum Master). She helps the team break the project down into **iterations**, tracking their progress on a **task board** and using project **velocity** and **burndown charts**—line charts that track the amount of work left on the project on a daily basis, "burning" down to zero when the work is done—to keep everyone up to date. This is the first time the team has actually been interested in what their project manager is doing, and it makes an improvement in how the work progresses.

Tom also wanted to get in on the agile improvement. Dan, Bruce, and Joanna started calling him the **product owner**, and Tom began to write **user stories** for the team so they could have a better idea of what the software needed to do. He worked with them to build **release plans** based on the stories, and now he felt like he had direct control over what the team would build.

Best of all, Bruce started holding **daily standup meetings** with Joanna, Dan, and all of the programmers, and Tom started attending them as well. It was a little awkward at first, but by the time the project was rolling, everyone was very comfortable giving each other real feedback and honest assessments of how the project was going. Bruce convinced the team to start holding **retrospectives** together at the end of each iteration, and he was pleased to see the team genuinely trying to carry out the improvements that they talked about during the retrospectives.

"Better-Than-Not-Doing-It" Results

It all worked. The team improved, and the project got better...up to a point.

By "going agile," everyone on the team got better at their jobs. Dan and the developers started to develop better habits and write better code. Joanna had a better handle on where the project was at any time. Tom communicated with the team much more, and that gave him better control over the software that they built, which meant he could do a better job of delivering what the users needed. Bruce could concentrate on improving his team's skills and communication.

But have they *really* become an agile team?

They adopted a lot of great practices. Many of those practices were better versions of what they had previously been doing, and all of them made each person more productive at their job. And that was definitely an improvement.

But while the team was happier, and the jukebox project really was going better than any of their previous projects, they also had some reservations about the new, more agile world they found themselves inhabiting. Dan, for instance, thinks that the team is definitely building better code now than they were before, but he finds himself making some technical sacrifices to meet the schedule.

Joanna is happier with the fact that she has some control over the way the project is run. But breaking the project down into short iterations makes her feel a bit blind. Instead of having a big, top-down schedule that she can use as a roadmap, she now finds herself increasingly depending on the team to tell her what's going on at the daily standups. The daily standups are useful, but they mainly consist of each team member reciting his or her status—which Joanna dutifully writes down and communicates to the stakeholders. It's starting to feel more like she's just a coordinator or organizer than a project manager in control of a project. She only focuses on status now, and that makes it difficult for her to spot obstacles and remove them for the team. The team is better at reacting to change, but that puts Joanna in a somewhat uncomfortable position where that's all she's focusing on: reacting, rather than planning.

Now that Tom's a product owner, he's thrilled with his increased ability to define what the team builds. But he's also torn, because he feels like he's expected to work for the team full time. He has to attend these daily meetings, and constantly respond to emails and questions from developers who ask about details of the software they're building. Sometimes they ask questions that he doesn't know the answer to, and other times he wishes they could just answer those questions themselves. He already has a job, and he doesn't feel like the others are meeting him halfway—he feels like they're pushing all of the responsibility for building a great product back on him, and he doesn't have all of the answers. After all, his "real" job is account manager—those jukeboxes won't sell themselves. How can he stay up to date on his accounts and what his users need if he's spending all of his time answering questions for the programmers?

As for Bruce, he's happy that the team is delivering more often. But when he takes a step back and really looks at what's going on, something seems unsatisfying and incomplete, and he's not sure why. Clearly this is an improvement, considering most of his previous projects felt like they were a hair's breadth from failure. It seems to Bruce like the agile adoption made things better, cut down on the personal heroics, and reduced the number of long nights and weekends of work. But he also feels like going agile brought its own set of problems.

It's not uncommon for team members, and especially team leads, to feel the way Bruce does: a little disappointed after their first attempt at agile adoption. The blogs and books they've read and the training they've attended talked about "astonishing results" and "hyper-productive teams." This team feels like the jukebox project is an improvement on their previous projects, but they definitely don't feel "hyper-productive," and nobody's really been astonished at the results.

There's a general feeling that the project has gone from dysfunctional to functional, and that's very good. They've gotten what we like to call **better-than-not-doing-it results**. But is this really all there is to agile?